

PPGEEC2327 – Tópicos Especiais em Processamento Inteligente da Informação

Inteligência artificial aplicada à geofísica

Capítulo 4 — Introdução às redes neurais

Baseado em Schuster, G.T. — *Machine Learning Methods in Geoscience*

Professor: Tiago Barros

Programa de Pós-graduação em Engenharia Elétrica e de Computação (PPgEEC) — UFRN

Sumário da Aula

Introdução: Regressão vs. Classificação

Inspiração Biológica

Rede Neural Geral

Interpretação Geométrica e Universalidade

Workflow, Feed-forward e Backpropagation

Funções de Ativação

Função Objetivo por Entropia Cruzada

Tipos de Classificação

Boas Práticas

Síntese e Fechamento

Introdução: Regressão vs. Classificação

De Mínimos Quadrados à Classificação

- Cap. 2: coeficientes ótimos de $\mathbf{W}$ que minimizam $\|\mathbf{W}\mathbf{x} - \mathbf{t}\|^2$ — $t \in \mathbb{R}$ (**regressão**).
- Agora: **classificação** — os elementos de $\mathbf{t}$ (ou $\mathbf{y}$) são restritos a um conjunto pequeno de rótulos, um por categoria.

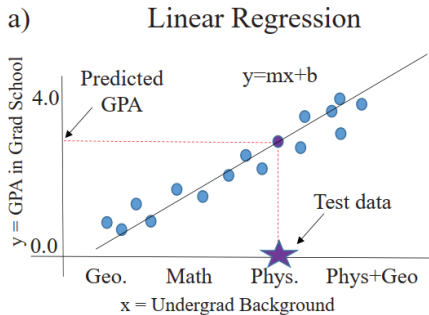
Exemplo

Classificação binária de motocicletas vermelhas: $t = 1$ (vermelha) ou $t = 0$ (não vermelha) — apenas dois valores possíveis, ao contrário da regressão.

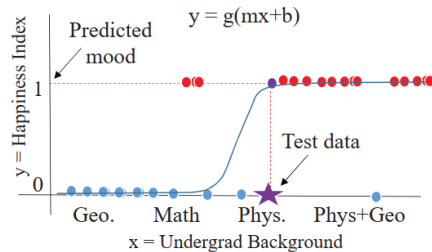
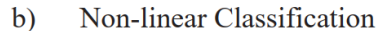
Judea Pearl (Prêmio Turing 2018)

“Todas as conquistas impressionantes do *deep learning* equivalem a apenas ajuste de curvas.”

De Mínimos Quadrados à Classificação



Mary has a physics BS degree, what is her expected GPA in graduate school?



Will Mary be happy in the geoscience graduate school?

Regressão Linear vs. Classificação Não Linear

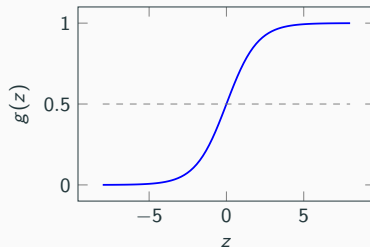
Regressão: $y = mx + b$

- Saída contínua (ex.: GPA 0 a 4.0).

Classificação: $y = g(mx + b)$

- Saída binária ajustada por uma **sigmoide** S-shaped.
- $g(z) = \frac{1}{1 + e^{-z}}$ (função logística).

Sigmoide: função de “squashing”



Definição

A função objetivo de mínimos quadrados para classificação:

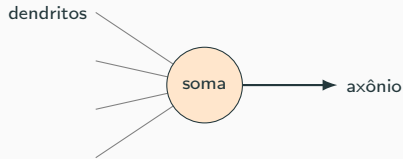
$$\epsilon = \frac{1}{2} \sum_{k=1}^K (g(mx^{(k)} + b) - y^{(k)})^2$$

Inspiração Biológica

Inspiração Biológica do Neurônio

Modelo de McCulloch & Pitts (1943): soma ponderada de sinais + limiar.

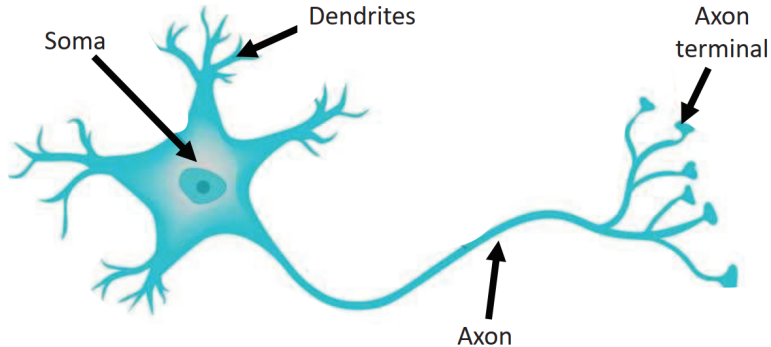
- **Dendritos:** “fios” de entrada; peso $\propto$ concentração de neurotransmissores.
- **Soma:** soma ponderada dos sinais de entrada (operação matriz-vetor).
- **Axônio:** dispara sinal “tudo ou nada” se exceder um limiar.



Ideia-Chave

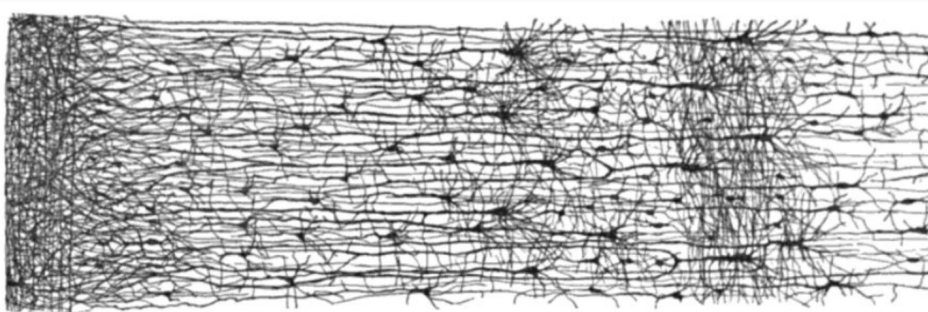
Beniaguev et al. (2021): os ramos dendríticos podem atuar como detectores de padrões espaciais — um único neurônio cortical se comporta como uma rede profunda de **5 camadas**.

Neuron



Inspiração Biológica do Neurônio

- O cérebro humano médio tem aproximadamente 86 bilhões de neurônios.
- Cada neurônio está tipicamente conectado a milhares de outros neurônios.
- Os neurônios são frequentemente dispostos em camadas consecutivas (por exemplo, no córtex cerebral)



Rede Neural Geral

Definição de uma Rede Neural

Definição

Uma rede neural (NN) é um modelo matemático não linear

$$f(\mathbf{x}^{(k)}, \mathbf{W}) \approx \mathbf{y}^{(k)}$$

que mapeia a entrada $\mathbf{x}^{(k)}$ para a saída esperada $\mathbf{y}^{(k)}$, com $\mathbf{W}$ os parâmetros a determinar.

- Exemplo simples: $\mathbf{W} = (b, m)$, $\mathbf{x} = (1, x) \Rightarrow \sigma(\mathbf{w}^T \mathbf{x}) = \sigma(mx + b)$.
- **Matriz densa** $\mathbf{W} \rightarrow$ rede totalmente conectada (*fully-connected*, FCNN) — foco deste e dos próximos 2 capítulos.
- **Matriz esparsa** (convolução) $\rightarrow$ rede convolucional (CNN) — Caps. 9–12.

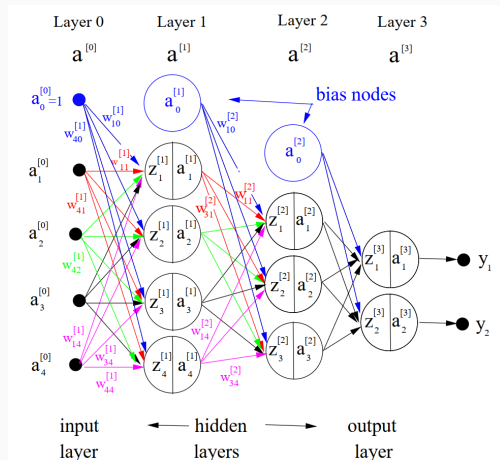
Arquitetura de uma Rede Multicamada

Duas operações em cada nó:

$$z_i^{[n]} = \sum_{j=1}^J w_{ij}^{[n]} a_j^{[n-1]} \quad a_i^{[n]} = g(z_i^{[n]})$$

Objetivo: encontrar $w_{ij}^{[n]}$ que minimizam

$$\epsilon = \frac{1}{2} \sum_i \sum_k (g(z_i^{[N](k)}) - y_i^{(k)})^2 + \lambda (\text{regularização})$$



Interpretação Geométrica e Universalidade

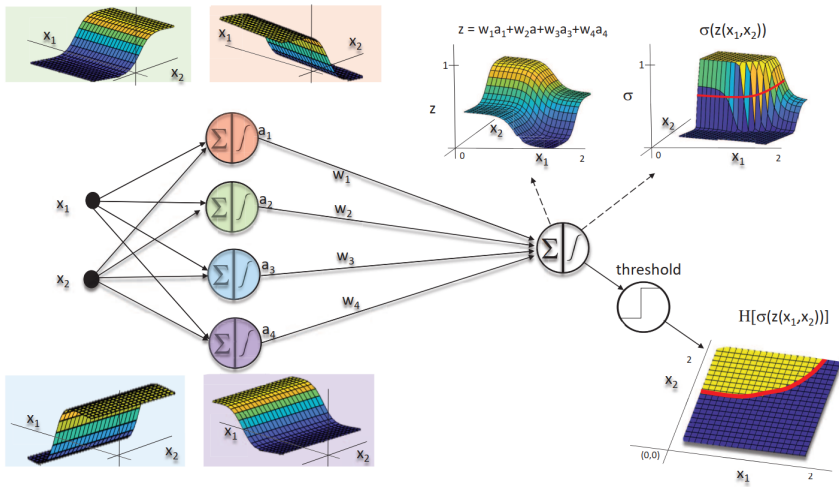
Interpretação Geométrica: Soma de Sigmoides

- Cada nó oculto $\rightarrow$ uma sigmoide orientada de forma diferente no espaço x_1 – x_2 .
- A saída é uma soma ponderada dessas sigmoides (funções de base).
- O limiar $\sigma(z) = 0.5$ define a **fronteira de decisão**.
- Mais nós/camadas $\rightarrow$ fronteiras de forma quase arbitrária.

Ideia-Chave

Soma de 4+ sigmoides (Fig. 4.7–4.9 do texto) pode reconstruir curvas “em ilha” de forma quase arbitrária — inclusive uma gaussiana.

Interpretação Geométrica: Soma de Sigmoides



Teorema de Kolmogorov: Universalidade

Qualquer função contínua $f(\mathbf{x})$ pode ser escrita como:

$$f(\mathbf{x}) = \sum_{j=1}^{2n+1} \tilde{\sigma} \left(\sum_{i=1}^d \psi_{ij}(x_i) \right)$$

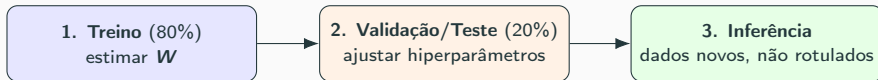
- d : dimensão de $\mathbf{x}$; ψ e $\tilde{\sigma}$: funções apropriadas.
- Estrutura análoga a uma rede neural de 2+ camadas com $\tilde{\sigma} \rightarrow$ sigmoide.

Implicação prática

Uma NN com 2+ camadas e nós suficientes por camada pode, em princípio, reconstruir qualquer mapeamento contínuo $f(\mathbf{x}, \mathbf{W}) = \mathbf{y}$. Na prática, são necessários **muitos nós** e **mais de duas camadas** para bons resultados em dados reais.

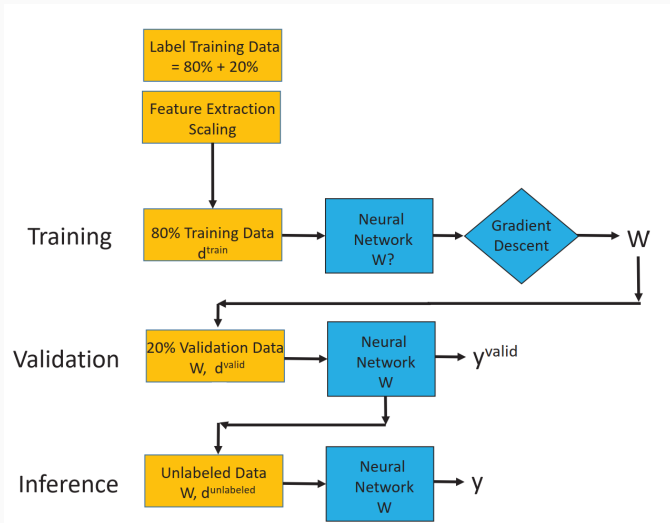
Workflow, Feed-forward e Backpropagation

Fluxo de Trabalho da Rede Neural Supervisionada



- Regra prática (Ng & Jordan, 2011): número de exemplos de treino $\gtrsim$ número de parâmetros desconhecidos em W .
- Otimização: método de descida do gradiente (Caps. 2–3).

Fluxo de Trabalho da Rede Neural Supervisionada



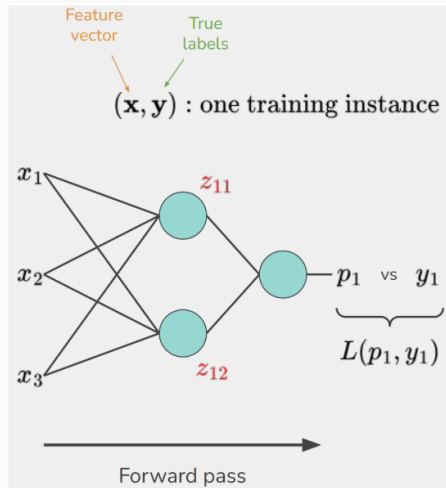
Fórmula Recursiva de *Feed-forward*

$$\mathbf{y}^{\text{pred.}} = g(\mathbf{W}^{[N]}\mathbf{a}^{[N-1]}) = g\left(\mathbf{W}^{[N]}g(\mathbf{W}^{[N-1]}\dots g(\mathbf{W}^{[1]}\mathbf{a}^{[0]})\dots)\right)$$

- Iniciar em $\mathbf{a}^{[0]} = \mathbf{x}$, calcular $g(\mathbf{W}^{[1]}\mathbf{a}^{[0]})$, reutilizar como entrada da camada seguinte.
- Repetir até a última camada $\Rightarrow$ predição $\mathbf{y}^{\text{pred.}}$.
- Custo eficiente: reaproveitamento de resultados intermediários (Cap. 6 detalha a implementação matricial).

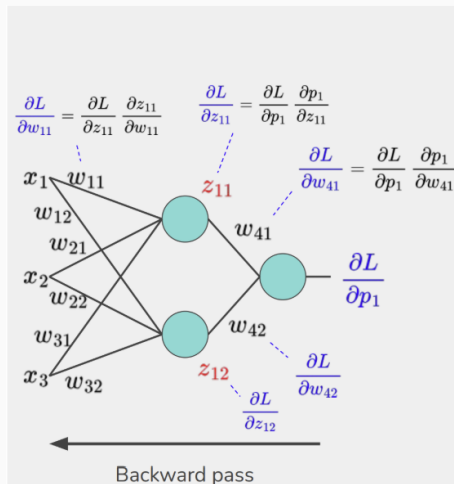
Forward propagation (mais de 1 camada)

- Os gradientes são calculados passando cada instância de treinamento para a frente (forward) e para trás (backward) através da rede.
- Passagem para a Frente (Forward pass):** a rede prevê uma instância de treinamento, enquanto armazena as saídas de todos os neurônios ocultos.



Backward propagation (mais de 1 camada)

- Os gradientes são calculados passando cada instância de treinamento para a frente (forward) e para trás (backward) através da rede.
- Passagem para Trás (Backward pass):** calcular a derivada da perda em relação às saídas da rede e, em seguida, propagar essas derivadas sucessivamente para as camadas anteriores, usando a regra da cadeia, a fim de calcular as outras derivadas.



Backpropagation: Analogia com Migração Sísmica

- *Feed-forward*: entrada $\rightarrow$ camadas profundas (como propagação de onda direta).
- *Backpropagation*: resíduo de saída retropropagado da última camada até a entrada.

Paralelo com migração por mínimos quadrados (LSM)

Em LSM (Schuster, 2017), o resíduo de dados sísmicos $\Delta \mathbf{d}$ é retropropagado da superfície para camadas profundas via $\mathbf{m} = \mathbf{L}^T \Delta \mathbf{d}$, onde $\mathbf{L}^T$ é o operador de migração (adjunto do operador de modelagem direta $\mathbf{L}$).

A retropropagação de erro em redes neurais é, matematicamente, o mesmo tipo de operação adjunta.

Funções de Ativação

Ideia-Chave

A sigmoide satisfaz as propriedades de uma distribuição de probabilidade binária: $0 \leq g(z) \leq 1$ e soma 1 para $y = 0$ e $y = 1$.

$$p(y = 1 | \mathbf{x}, \mathbf{w}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}, \quad p(y = 0 | \mathbf{x}, \mathbf{w}) = \frac{1}{1 + e^{+\mathbf{w}^T \mathbf{x}}}$$

Problema do gradiente que desaparece

Em redes profundas (14+ camadas), dg/dz retropropagado tende a zero para $|z| > 0 \rightarrow$ atualização mínima nas camadas profundas (Bengio et al., 1994).

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{LReLU}(z) = \max(\alpha z, z), \quad 0 < \alpha \leq 1$$

$$\text{Softplus}(z) = \ln(1 + e^z)$$

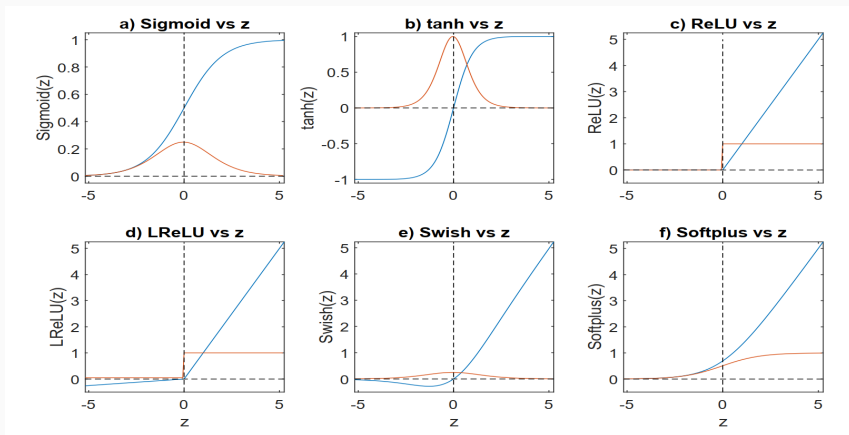
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\text{Swish}(z) = \frac{z}{1 + e^{-z}}$$

Ideia-Chave

Goodfellow et al. (2016): “Em redes modernas, a recomendação padrão é usar ReLU.”
Gradiente constante para $z > 0 \rightarrow$ remedia o desaparecimento do gradiente.

Funções de Ativação Modernas



Função Objetivo por Entropia Cruzada

Entropia Cruzada: Motivação Probabilística

A sigmoide como função de verossimilhança para y binário:

$$p(y | \mathbf{x}, \mathbf{w}) = (g(\mathbf{w}^T \mathbf{x}))^y (1 - g(\mathbf{w}^T \mathbf{x}))^{1-y}$$

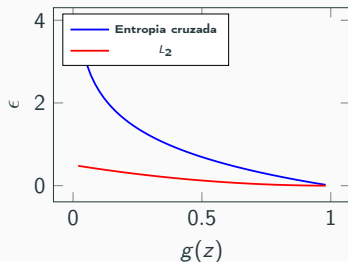
Maximizar a verossimilhança $\equiv$ minimizar a **entropia cruzada**:

$$\epsilon = -y \ln g(\mathbf{w}^T \mathbf{x}) - (1 - y) \ln(1 - g(\mathbf{w}^T \mathbf{x}))$$

Para K pares de treino:

$$\epsilon = -\frac{1}{K} \sum_{k=1}^K \left[y^{(k)} \ln g(\mathbf{w}^T \mathbf{x}^{(k)}) + (1 - y^{(k)}) \ln(1 - g(\mathbf{w}^T \mathbf{x}^{(k)})) \right]$$

Por que Entropia Cruzada Converte Mais Rápido?



- $-\ln g(z)$ cresce muito mais **acentuadamente** que $\frac{1}{2}(g(z) - 1)^2$ perto de $g(z) = 0$.
- Gradientes maiores $\rightarrow$ convergência mais rápida em problemas de classificação.
- Minimizar entropia cruzada para regressão logística é um problema **convexo** (Loiseau, 2020) $\rightarrow$ único mínimo global.

Tipos de Classificação

Três Tipos de Classificação

Binária

$C = 2$, 1 nó de saída, $y \in \{0, 1\}$.

Multirrótulo

$C > 2$, rótulos **independentes**;
vários $y_c = 1$ simultaneamente (ex.:
“tem olhos” E “tem língua”).

Multiclasse

$C > 2$, codificação **1-de- C** : apenas
um $y_c = 1$ (ex.: gato **ou** cachorro **ou**
nenhum).

Multirrótulo — entropia cruzada somada sobre classes independentes:

$$\epsilon = -\frac{1}{K} \sum_{c=1}^C \sum_{k=1}^K \left[y_c^{(k)} \ln g_c(\mathbf{w}^T \mathbf{x}^{(k)}) + (1 - y_c^{(k)}) \ln(1 - g_c(\mathbf{w}^T \mathbf{x}^{(k)})) \right]$$

Multiclasse: A Função *Softmax*

- Função objetivo (entropia cruzada multiclasse):

$$\epsilon = - \sum_{c=1}^C \sum_{k=1}^K y_c^{(k)} \ln p(y_c^{(k)} | \mathbf{x}^{(k)}, \mathbf{W})$$

Exemplo

3 classes, alvo real (0, 0, 1): se $p_3 = 0.01 \Rightarrow \epsilon = 4$ (ruim); se $p_3 = 0.99 \Rightarrow \epsilon \approx 0.001$ (quase o mínimo).

Boas Práticas

Boas Práticas (Parte 1): Preparação e Inicialização

1. **Escalonamento dos dados:** centralizar e normalizar (variância 1) ou mapear para $[0, 1]$.
2. **Inicialização dos pesos:** aleatória em $[-1, 1]$.
 - **Xavier** (Glorot & Bengio, 2010): $\pm 1/\sqrt{n}$, n = nós da camada anterior.
 - **He** (He et al., 2015), para ReLU: $\pm \sqrt{2/n}$.
3. **Inicialização do viés:** valor pequeno (ex.: 0.1) para ReLU $\rightarrow$ mantém unidades ativas inicialmente.
4. **Funções de ativação:** preferir ReLU/Swish nas camadas ocultas (convergência mais rápida que sigmoide).

Boas Práticas (Parte 2): Arquitetura e Regularização

5. **CNNs**: preferir CNNs em relação a FCNNs quando a rede tem mais de 5 camadas (Cap. 9).
6. **Rótulo “desconhecido”**: em classificação multirrótulo, sempre incluir uma classe “*unknown*” para mitigar erro em exemplos fora da distribuição de treino.
7. **Regularização**: penalidade no objetivo, redução de pesos, *early stopping*, *dropout* (Caps. 9, 14), ruído na entrada.

Ideia-Chave

Objetivo da regularização: evitar **sobreajuste** — soluções menos complexas que generalizam melhor.

Mini-Lotes e Descida Estocástica do Gradiente (SGD)

8. Mini-lotes (*Mini-batch*): Para economizar memória ao calcular o gradiente, os dados são divididos em B mini-lotes que não se sobrepõem, e a solução é atualizada sequencialmente, dando passos ao longo dos gradientes de cada mini-lote. Procedimento chamado de descida do gradiente estocástica (SGD).

```
1  for it = 1:nit                                % laco sobre epocas
2      for ib = 1:B                              % laco sobre B mini-lotes
3          w{ib+1} = w{ib} - alpha*grad_eps{ib};
4      end
5      w{1} = w{B+1};                            % reembaralhar dados em B mini-lotes
6  end
```

- Uma passada completa por todos os B lotes = **1 época**.
- Tamanho de lote típico: **50–256** exemplos (Ruder, 2017).
- Converge ao mínimo global para funções convexas; ao mínimo local para não convexas.

- 10. Comprimento de passo:** métodos adaptativos (momentum, Adam, Adagrad — Cap. 3) são populares; reduzir o passo perto do mínimo evita ultrapassagem.
- 11. Verificação do gradiente:** comparar o gradiente do código com a estimativa por diferenças finitas:

$$\left[\frac{\partial \epsilon(\mathbf{w})}{\partial w_i} \right]^{\text{DF}} = \frac{\epsilon(\mathbf{w} + \delta w_i \hat{\mathbf{e}}_i) - \epsilon(\mathbf{w})}{\delta w_i}$$

Ideia-Chave

Alternativa: usar bibliotecas de **diferenciação automática** (Cap. 26) para checar a implementação do gradiente.

Síntese e Fechamento

Resumo do Capítulo

- Uma NN mapeia $\mathbf{x} \rightarrow \mathbf{y}$ por uma sequência concatenada de: (1) multiplicação matriz-vetor e (2) limiar não linear (ativação).
- Mais camadas/nós $\rightarrow$ maior capacidade de modelar mapeamentos complexos (Kolmogorov: universalidade teórica).
- Matrizes densas (FCNN) $\rightarrow$ custo maior que matrizes esparsas (CNN, Cap. 9).
- *Feed-forward* (entrada $\rightarrow$ saída) e *backpropagation* (saída $\rightarrow$ entrada) são operações **adjuntas** uma da outra.
- Entropia cruzada geralmente converge mais rápido que L_2 para classificação; *softmax* generaliza a sigmoide para multiclasse.
- Boas práticas (escalonamento, inicialização, regularização, mini-lotes) são essenciais para treinamento estável.

Obrigado!

Perguntas e discussão